Лабораторна робота № 1

Кореляційний аналіз у задачах виявлення прихованих об’єктів

Мета заняття: вивчити теоретичні основи та обмеження класичного кореляційного аналізу. Опанувати математичний апарат та фізичний зміст функції взаємної кореляції. Набути прак-тичних навичок застосування ковзного кореляційного аналізу для виділення детермінованого корисного сигналу на тлі інтенсивного випадкового шуму.

Завдання до лабораторної роботи:

1. Розробити власного клієнта для сервера ехолота — мовою програмування на влас-ний вибір (Python необов'язковий). Клієнт повинен:
2. Отримати конфігурацію та еталонний імпульс.
3. Для кожної точки маршруту запитувати зашумлений сигнал.
4. Самостійно реалізувати обчислення функції взаємної кореляції R(k) між отриманим сигналом і еталонним імпульсом. Використовувати готові бібліотечні функції зі спеціалізованих бібліотек(на кшталт numpy, tensorflow тощо) не можна — мета робо-ти полягає саме у принципах в реалізації самого підходу, а не у виклику чужих гото-вих функцій.
5. Знайти зсув kmax, що відповідає максимуму R(k).
6. Перевести знайдений зсув у глибину.
7. Пройти весь маршрут (усі кроки 0 .. total_steps-1) і вивести/відобразити отриманий профіль дна візуально (обмежень на використання графічних бібліотек немає).
8. Розрахувати максимальну глибину вимірювання системи.
9. Перевірити якість розрахунку глибини при усереднинні 3-4 запитів на кожен

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.125.16.000 – Лр.1		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Олійник Г. І.			Звіт з лабораторної роботи №1	Літ.	Арк. 1 / Аркушів 14
Перевір.		Петросян Р.В.					
Реценз.						ФІКТ, гр. ІПЗ-24-2	
Н. Контр.							
Зав.каф.							

крок (придушення хибних піків накопиченням). Усереднити отримані данні перед обчис-ленням кореляції (ефект можна показати, порівнявши результат з одним запитом і з накопиченням декількох).

Хід роботи

1. Створення власного клієнта для сервера ехолота

Для отримання інформації з ехолота розроблено клієнт на Python. Він звертається до сервера за адресою https://ooi-lab1.up.railway.app і використовує три ендпоінти:

Ендпоінт	Що повертає
GET /api/config	параметри ехолота: 1.кількість точок, 2. частота дискретизації, 3. швидкість звуку, 4. несуча частота
GET /api/pulse	відліки еталонного імпульсу
GET /api/sensor?step=N	зашумлений сигнал для точки маршруту N

Код запитів зроблено на стандартному urllib. Усі запити йдуть через одну функцію fetch_json(). Якщо запит не вдався, вона пробує ще до 5 разів, щоб випадковий збій мережі не зупиняв проходження маршруту.

Лістинг коду:

```
import json
import urllib.request

BASE_URL = "https://ooi-lab1.up.railway.app"

def fetch_json(path):
    for attempt in range(5):
        try:
            with urllib.request.urlopen(BASE_URL + path, timeout=30) as response:
                return json.loads(response.read().decode("utf-8"))
        except Exception:
            if attempt == 4:
```

```
                raise
        return None

    def get_config():
        return fetch_json("/api/config")

    def get_pulse():
        return fetch_json("/api/pulse")

    def get_sensor(step):
        return fetch_json("/api/sensor?step=%d" % step)
```

Код розбитий на кілька файлів: sonar_core.py (запити й усі розрахунки), sonar_gui.py (головне вікно з графіками), sonar_point.py (вікно аналізу однієї точки) і main.py (запуск).

2. Отримання конфігурації та еталонного імпульсу

При запуску програма викликає get_config() і get_pulse(). Перша функція повертає параметри ехолота, а друга, відліки еталонного зондувального імпульсу. Отримані параметри показані в панелі “Отримано від сервера” (Рис. 1).



Рис. 1. Отримана конфігурація ехолота

Отримані параметри:

Параметр	Значення
Кількість точок маршруту	100
Частота дискретизації	2000 Гц
Швидкість звуку	1500 м/с
Несуча частота	400 Гц
Довжина еталонного імпульсу	40 відліків
Довжина запису сигналу	1000 відліків

Еталонний імпульс складається з 40 відліків і є дискретизованим тональним сигналом із несучою частотою 400 Гц. За частоти дискретизації 2000 Гц на один період припадає 5 відліків, тому весь імпульс містить 8 періодів (Рис. 2).

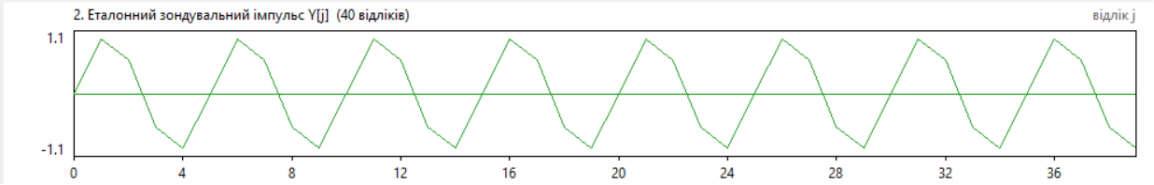


Рис. 2. Еталонний зондувальний імпульс

3. Запит зашумленого сигналу для кожної точки маршруту

Для кожної точки маршруту клієнт робить запит GET /api/sensor?step=N, де step - номер точки від 0 до 99. У відповідь сервер повертає масив із 1000 відліків прийнятого сигналу. Це відлуння від дна: той самий еталонний імпульс, але із затримкою, до якого додано випадковий шум. Чим глибше дно, тим більша затримка. Через шум відлуння на графіку не видно, тому далі сигнал порівнюється з еталонним імпульсом за допомогою кореляції.

Для усереднення потрібно кілька сигналів однієї точки, тому додано функцію fetch_signals(), яка робить кілька запитів одночасно, кожен у своєму потоці.

Лістинг коду:

```
def fetch_signals(step, count):
    if count == 1:
        return [get_sensor(step)]

    results = [None] * count
    errors = []

    def worker(index):
        try:
            results[index] = get_sensor(step)
        except Exception as error:
            errors.append(error)

    threads = [threading.Thread(target=worker, args=(i,)) for i in range(count)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
```

```
    if errors:
        raise errors[0]

    return results
```

Якщо count = 1, робиться один запит, інакше створюється count потоків. Сигнал для поточної точки відображається на графіку “Сигнал датчика X[i]” (Рис. 3): червона лінія - один запит, синя - усереднення чотирьох.

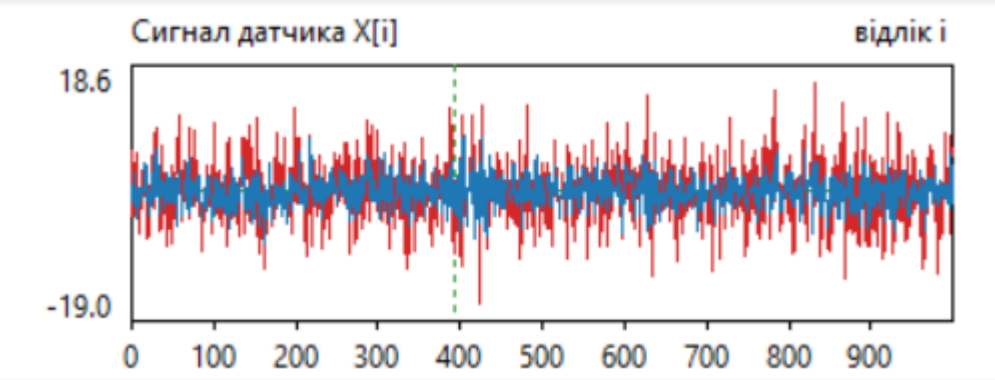


Рис. 3. Графік прийнятого сигналу датчика для поточної точки

4. Самостійна реалізація взаємної кореляції

Реалізовано кореляційний аналіз за формулою:

$$R(k) = \sum_i X[i] \cdot Y[i-k]$$

де X - прийнятий сигнал довжиною N, Y - еталонний імпульс довжиною M, k - зсув. Сума береться за тими відліками i, на яких імпульс накладається на сигнал. Еталонний імпульс по черзі зсувається вздовж сигналу і для кожного зсуву перемножуються відповідні відліки й додаються.

У програмі сума записана через j = i - k, де j - номер відліку імпульсу: R(k) = ∑ X[k + j] * Y[j], j = 0 … M-1

Лістинг коду:

```
def cross_correlation(signal, template):
    n = len(signal)
    m = len(template)

    shifts = n - m + 1
    result = [0.0] * shifts

    for k in range(shifts):
```

```
            total = 0.0
            for j in range(m):
                total += signal[k + j] * template[j]

            result[k] = total

        return result
```

Як працює функція:

1. shifts = n - m + 1 = 961 – кількість можливих зсувів (імпульс не виходить за межі сигналу), k від 0 до 960.

2. Зовнішній цикл перебирає зсуви k, внутрішній множить 40 відліків сигналу на відліки імпульсу й накопичує суму.

3. Результат кожного зсуву записується в result[k].

Там, де під імпульсом лише шум, R(k) невелике й хаотично коливається навколо нуля. Там, де імпульс накладається на відлуння, добутки мають однаковий знак і R(k) різко зростає, це є пік (Рис. 4).

5. Пошук зсуву k_max

Положення максимуму R(k) знаходить функція argmax(), яка проходить по масиву вручну, без max() та .index().

Лістинг коду:

```
def argmax(values):
    best_index = 0
    best_value = values[0]

    for i in range(1, len(values)):
        if values[i] > best_value:
            best_value = values[i]
            best_index = i

    return best_index, best_value
```

Функція повертає два числа: індекс k_max (потрібний зсув) і значення піку R(k_max).

Для точки step = 99 максимум R(k) для усередненого сигналу знайдено при k_max = 394, значення піку R(k_max) = 83,8 (Рис. 4).



Рис. 4. Функція взаємної кореляції R(k) та знайдений k_max

6. Переведення зсуву в глибину

Знайдений зсув k_max це номер відліку, з якого починається відлуння. Щоб отримати з нього глибину, спочатку зсув переводиться в час затримки, а потім у відстань. Для цього використовуються формули з теоретичних відомостей.

$$T_{\text{затримки}} = \frac{k_{max}}{F_s} \quad H = \frac{T_{\text{затримки}} \cdot V}{2}$$

Лістинг коду:

```
def shift_to_delay(k, sample_rate):
    return k / sample_rate

def shift_to_depth(k, sample_rate, sound_speed):
    return (k / sample_rate) * sound_speed / 2.0
```

Функція shift_to_delay() ділить зсув на частоту дискретизації (час затримки), а shift_to_depth() додатково множить час на швидкість звуку й ділить навпіл. Приклад для точки step = 99, де k_max = 394:

T = 394 / 2000 = 0,197 с = 197 мс

H = 0,197 * 1500 / 2 = 147,75 м

Програма виводить ті самі значення в панелі “Поточний замір”: затримка 197,00 мс і глибина 147,75 м (Рис. 5). Для одиночного запиту в цій точці отримано той самий зсув k_max = 394 і ту саму глибину.



Рис. 5. Затримка й глибина для точки step = 99

Для перевірки алгоритм застосовано до кількох точок маршруту (вікно “Аналіз точки”, усереднення чотирьох запитів):

Крок	k_max	Затримка, мс	Глибина, м
0	441	220,50	165,38
25	422	211,00	158,25
50	293	146,50	109,88
75	640	320,00	240,00
99	394	197,00	147,75

Глибина в кожній точці відповідає формулі H = k_max * 0,375 м. За всім маршрутом вона змінюється від 108,00 до 237,38 м.

На Рис. 6 показано вікно аналізу для точки step = 50: прийнятий сигнал, еталонний імпульс і функції кореляції для одного запиту та для усередненого сигналу.



Рис. 6. Аналіз точки step = 50 (червона лінія - один запит, синя - усереднення чотирьох запитів)

7. Проходження маршруту та профіль дна

Для побудови профілю програма проходить усі кроки маршруту від 0 до 99.

На кожному кроці робиться чотири запити, сигнали усереднюються, обчислюється кореляція, знаходиться k_max і розраховується глибина. Це виконує метод step_once():

Лістинг коду:

```
def step_once(self):
    step = len(self.depths_avg)
    if step >= self.total_steps:
        return False

    self.status.configure(text="крок %d..." % step)
    self.root.update_idletasks()

    try:
        single, averaged = core.measure_pair(step, self.pulse, self.sample_rate,
                                             self.sound_speed, REPEATS)
    except Exception as error:
        self.status.configure(text="помилка зв'язку: %s" % error)
        return False

    self.last_single = single
    self.last_avg = averaged
    self.depths_single.append(single["depth"])
    self.depths_avg.append(averaged["depth"])
    self.rows.append({
        "k1": single["k_max"], "k4": averaged["k_max"],
        "snr1": single["snr"], "snr4": averaged["snr"],
        "noise1": single["noise"], "noise4": averaged["noise"],
        "peak4": averaged["peak"], "delay4": averaged["delay"],
    })

    self.update_readout()
    self.draw_all()
    return True
```

Кнопка “Пройти все” повторює цей метод для всіх кроків, а кнопка “Вперед” виконує один крок. Профіль малюється з такими осями: X - номер кроку, Y - глибина від 0 до 360 м. Червона лінія - один запит, синя - усереднення чотирьох (Рис. 7).

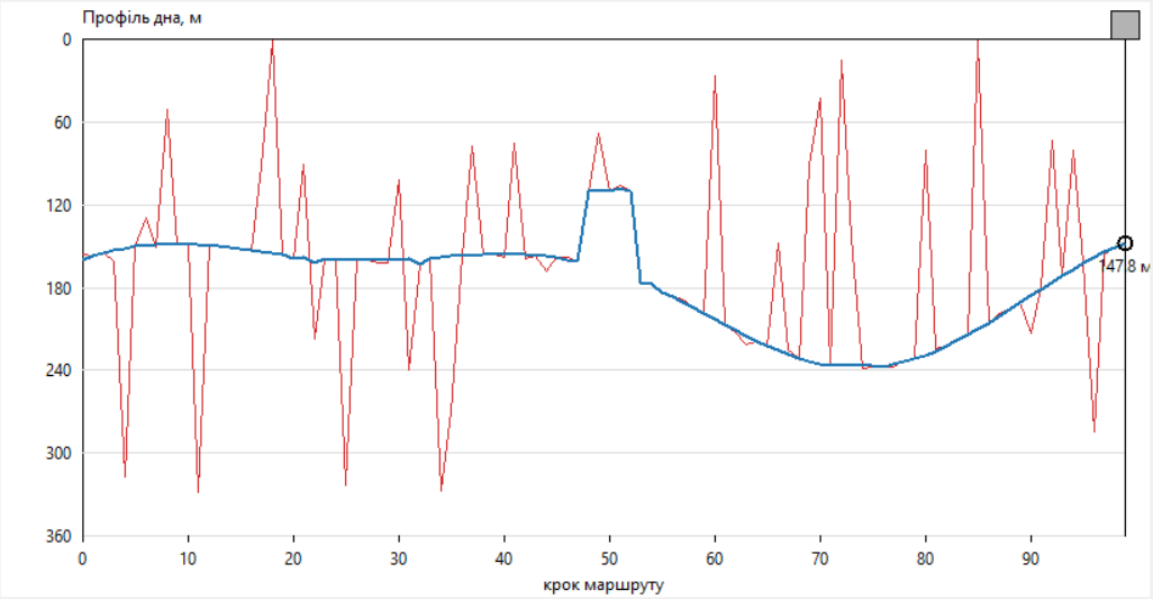


Рис. 7. Профіль дна вздовж маршруту

На усередненому профілі глибина до кроку 47 становить приблизно 150-160 м, на кроках 48-52 дно підіймається до близько 110 м, біля кроку 75 досягає максимуму близько 240 м, а в точці 99 становить 147,75 м. Мінімальна, максимальна та середня глибина - 108,00, 237,38 і 176,66 м (Рис. 8).



Рис. 8. Характеристики профілю дна

8. Максимальна глибина вимірювання

Імпульс довжиною M має повністю вміститися в запис довжиною N, тому максимальний зсув дорівнює N - M = 1000 - 40 = 960. Відповідно H_max = (960 / 2000) * 1500 / 2 = 360 м.

Окрім максимальної глибини, програма розраховує ще кілька характеристик системи (Рис. 9):

1. Тривалість імпульсу: 40 / 2000 = 20 мс.

2. Роздільна здатність 0,375 м - зміна глибини на один відлік зсуву.

Використовується також для допуску розбіжних кроків (2 відліки = 0,75 м).

3. Максимальна глибина 360 м - задає також масштаб осі профілю дна.

4. Кількість множень на один крок: (1000 - 40 + 1) * 40 = 38 440.

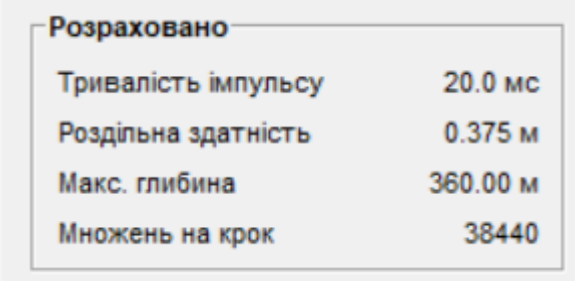


Рис. 9. Розрахована конфігурація ехолота

Лістинг коду:

```
def shift_to_depth(k, sample_rate, sound_speed):
	return (k / sample_rate) * sound_speed / 2.0

def pulse_duration(template_length, sample_rate):
	return template_length / sample_rate

def depth_resolution(sample_rate, sound_speed):
	return shift_to_depth(1, sample_rate, sound_speed)

def max_measurable_depth(signal_length, template_length, sample_rate, sound_speed):
	return shift_to_depth(signal_length - template_length, sample_rate,
sound_speed)

def multiplications_per_step(signal_length, template_length):
	return (signal_length - template_length + 1) * template_length
```

9. Усереднення сигналів

Через шум пік кореляції в одиночному вимірюванні іноді хибний. Тому для кожної точки робиться чотири запити, і сигнали усереднюються до обчислення кореляції: відлуння в запитах однакове й зберігається, а шум різний і слабшає. Усереднення виконує функція average_signals():

Лістинг коду:

```
def average_signals(signals):
    count = len(signals)
    length = len(signals[0])
```

```
    result = [0.0] * length

    for signal in signals:
        for i in range(length):
            result[i] += signal[i]

    for i in range(length):
        result[i] /= count

    return result
```

Функція додає відліки всіх сигналів по індексах і ділить суму на їх кількість. Для порівняння з одним запитом функція measure_pair() аналізує перший із чотирьох сигналів (один запит) і середній сигнал (усереднення), тобто обидва результати отримано з тих самих запитів:

Лістинг коду:

```
def measure_pair(step, template, sample_rate, sound_speed, repeats):
    signals = fetch_signals(step, repeats)

    single = analyse(signals[0], template, sample_rate, sound_speed)
    single["step"] = step
    single["repeats"] = 1

    averaged = analyse(average_signals(signals), template, sample_rate,
sound_speed)
    averaged["step"] = step
    averaged["repeats"] = repeats

    return single, averaged
```

Для оцінки якості програма виводить відношення пік/СКЗ, СКВ шуму, середній стрибок профілю між сусідніми кроками, кількість розбіжних кроків і кореляцію Пірсона між профілями. Показники для точки step = 99 наведено на Рис. 10 і в таблиці.



Рис. 10. Показники поточного заміру для точки step = 99

Показник	Один запит	Усереднення 4
Пік/СКЗ	4,17	6,52
СКВ шуму	4,554	2,332
Глибина, м	147,75	147,75

СКВ шуму зменшилось майже вдвічі, а пік/СКЗ зросло з 4,17 до 6,52.

На профілі (Рис. 7) червона лінія містить багато стрибків, а синя плавна: стрибок профілю зменшився з 58,83 до 3,27 м (приблизно у 18 разів), а на 50 кроках зі 100 одиночний результат відрізняється від усередненого більш ніж на 0,75 м (Рис. 8). Наприклад, у точці step = 50 (Рис. 6) один запит дав хибний пік k_max = 607 (227,62 м), а усереднення k_max = 293 (109,88 м).

Усереднення зменшує ймовірність хибного піку, але не усуває її повністю: у точках 0 і 25 результати відрізняються на 15 і 5 відліків, тобто шум перемикає вибір на сусідній максимум кореляції.

Посилання на github репозиторій - https://github.com/Hlib-Oliinyk/OOI.git

Висновки: у ході виконання лабораторної роботи розроблено клієнт для сервера ехолота, який отримує конфігурацію, еталонний імпульс і зашумлений сигнал для кожної зі 100 точок маршруту. Взаємну кореляцію R(k) реалізовано самостійно, без бібліотечних функцій. Для кожної точки знайдено k_max і розраховано глибину за

формулою H = k_max * V / (2 * Fs). Побудовано профіль дна вздовж усього маршруту: за усередненими даними глибина змінюється приблизно від 108 до 237 м. Максимальна глибина вимірювання становить 360 м, роздільна здатність - 0,375 м. Порівняння одного запиту з усередненням чотирьох показало, що усереднення зменшує СКВ шуму майже вдвічі (з 4,554 до 2,332), підвищує відношення пік/СКЗ (з 4,17 до 6,52) і зменшує кількість хибних піків: середній стрибок профілю зменшився з 58,83 до 3,27 м. Повністю хибні піки усереднення не усуває, але суттєво підвищує надійність визначення глибини.